

Introduction to React



JSX Basics

Syntax blending
HTML and JS



Components

Reusable UI building
blocks



State & Props

Manage data and
pass values



Lifecycle

Mounting, updating,
unmounting phases



Hooks

useState, useEffect
and custom hooks



Routing

Client navigation
with React Router



Performance

Optimise renders
and memoise



Testing

Unit and integration
testing patterns

FUNDAMENTALS

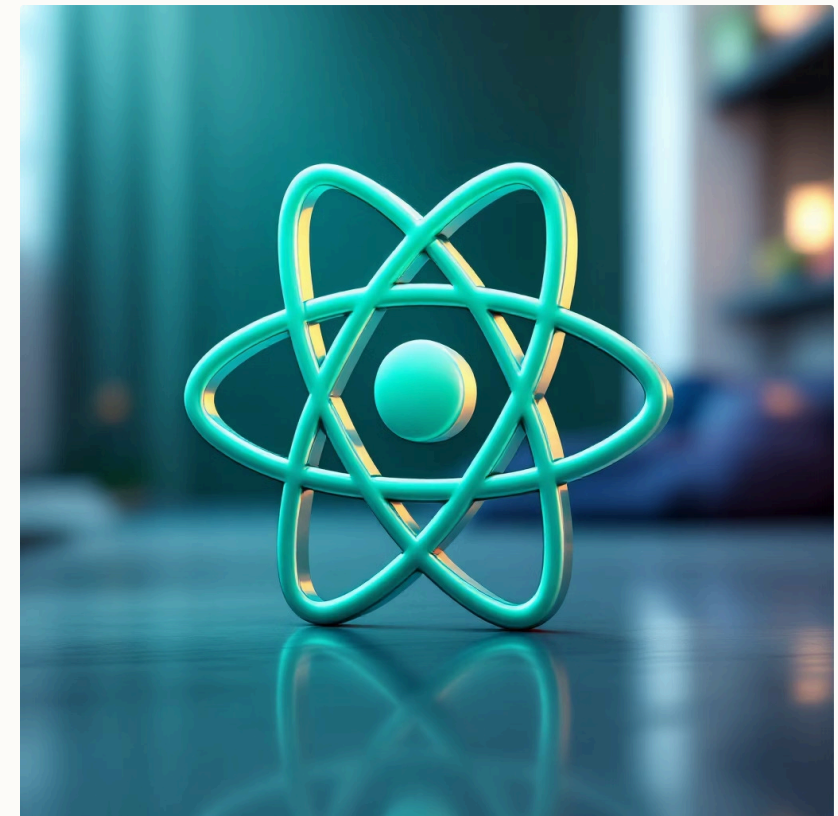
What is React?

React is an **open-source JavaScript library** developed and maintained by Facebook (Meta) for building user interfaces, particularly single-page applications. It was first released in 2013 and has since become one of the most popular front-end development tools in the industry.

React follows a **declarative programming paradigm**, allowing developers to design interactive UIs efficiently. Rather than manipulating the DOM directly, developers describe what the UI should look like, and React handles the updates automatically.

Why React is Used in Modern Web Development

- **Enhanced performance** through Virtual DOM implementation
- **Component reusability** reduces development time significantly
- **Strong community support** with extensive documentation and resources
- **SEO-friendly** applications with server-side rendering capabilities
- **Backed by Meta**, ensuring long-term stability and continuous improvement



- ❏ **Key Point:** React is a library, not a framework. It focuses specifically on the view layer of applications, giving developers flexibility in choosing other tools for state management and routing.

CORE CONCEPTS

Features of React

React's architecture introduces several innovative features that distinguish it from traditional web development approaches. Understanding these core features is essential for effective React development.

Component Model

Build UIs from reusable, isolated components

Unidirectional Data

Predictable state flow through props and state



Virtual DOM

Efficiently diff and update the UI for performance

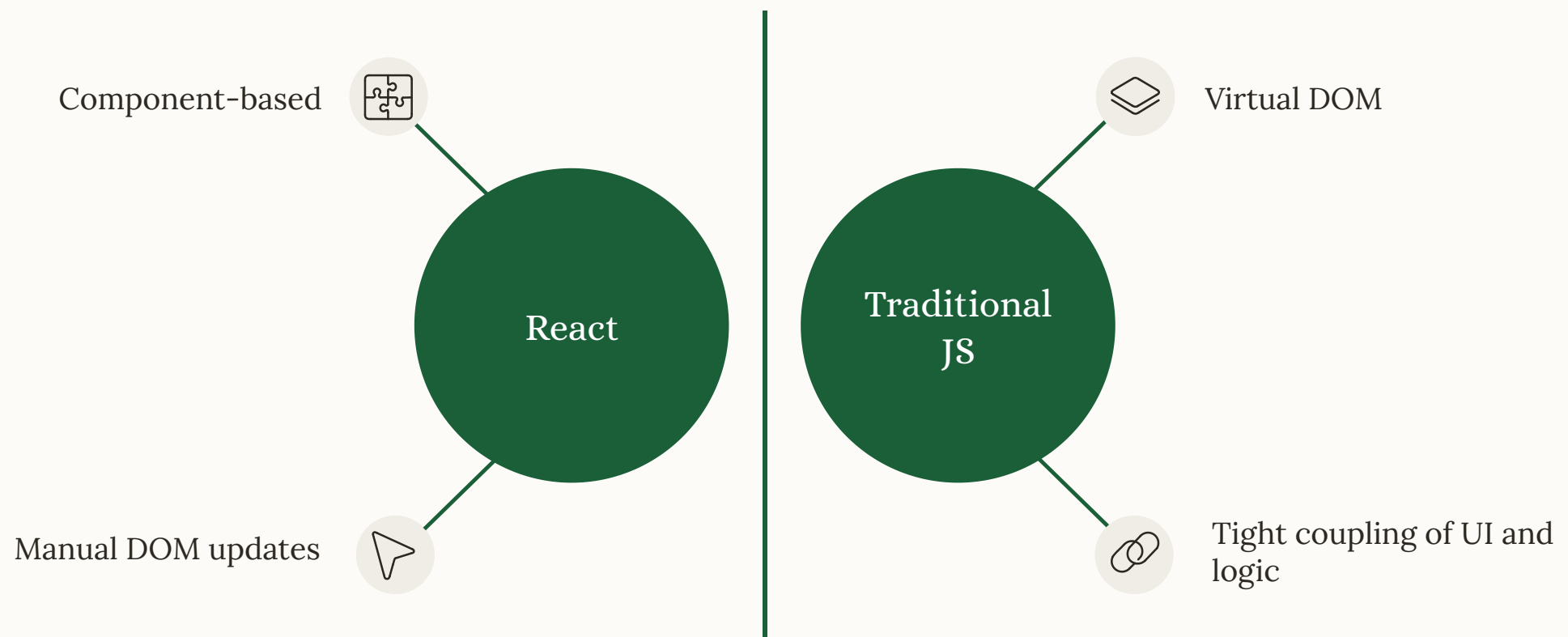
Hooks & APIs

Compose logic with hooks for reusable behaviour

COMPARISON

React vs Traditional JavaScript

Understanding the differences between React and traditional JavaScript approaches helps illustrate why React has become the preferred choice for modern web application development.



Efficiency and Scalability

React's architecture inherently supports **scalability**. As applications grow in complexity, React's component model and efficient update mechanisms ensure that performance remains consistent. The ecosystem includes tools like React Router for navigation and Redux for state management, facilitating the development of large-scale applications.

FUNDAMENTALS

Basic React Concepts

Mastering these foundational concepts is crucial for developing React applications effectively. These building blocks form the basis of all React development work.

01

Components

The fundamental building blocks of React applications. A component is a JavaScript function or class that optionally accepts inputs (props) and returns React elements describing what should appear on the screen.

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

03

Props (Properties)

Props are **read-only** arguments passed from parent to child components, similar to function parameters. They enable component reusability by allowing the same component to be used with different data.

```
function Greeting(props) {
  return <p>Hello, {props.username}</p>;
}
// Usage: <Greeting username="Student" />
```

02

JSX (JavaScript XML)

A syntax extension for JavaScript that allows writing HTML-like code within JavaScript. JSX makes component structure more readable and intuitive. Babel transpiles JSX into regular JavaScript function calls.

```
const element = <h1>Welcome to React</h1>;
// Transpiled to: React.createElement('h1', null, 'Welcome to React')
```

04

State

State is a built-in object that stores **dynamic data** specific to a component. Unlike props, state is managed within the component and can be updated, triggering re-renders. Modern React uses the `useState` hook for state management.

```
const [count, setCount] = useState(0);
// count: current state value
// setCount: function to update state
```

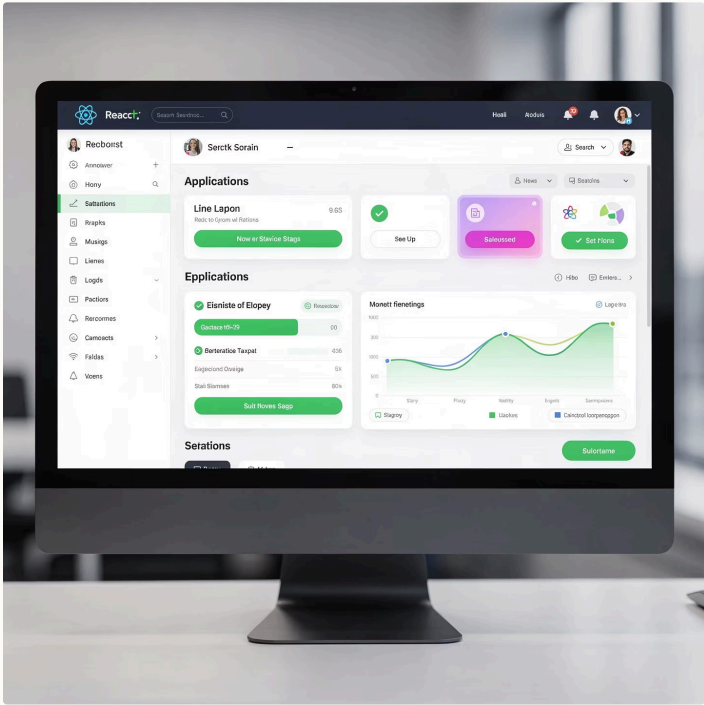
Summary & Learning Outcomes

Key Takeaways

- React is a powerful JavaScript library for building dynamic, component-based user interfaces
- Virtual DOM ensures optimal performance through efficient update mechanisms
- Component reusability and declarative syntax improve development efficiency
- JSX syntax combines the power of JavaScript with the familiarity of HTML markup
- Props enable data flow between components whilst state manages dynamic component data

Where React is Used

React powers numerous high-traffic applications including **Facebook**, **Instagram**, **Netflix**, **Airbnb**, **WhatsApp Web**, and **BBC**. It's extensively used in enterprise applications, progressive web apps, mobile applications (via React Native), and modern dashboard interfaces.



Expected Learning Outcomes

- Understand React's core philosophy and architecture
- Differentiate between props and state effectively
- Create basic functional components with JSX
- Explain Virtual DOM and its performance benefits
- Prepare for practical implementation in projects

